

// DO IT YOURSELF – PROJECT SERIES

DIY PROJECTS

Join the Community (Mandatorily) : [Link](#)

3

QUANT

2

CAPITAL

1

DE-FI

6

TOTAL PS

TABLE OF CONTENTS

01	REGIME-SHIFT	Quant • Easy
MACRO-AWARE TACTICAL ASSET ALLOCATION ENGINE		
02	VECM-ARB	Quant • Hard
DISTRIBUTED CROSS-ASSET STATISTICAL ARBITRAGE ENGINE		
03	NANOMATCH	Quant - Systems
ULTRA-LOW LATENCY ORDER MATCHING ENGINE		
04	COMMISSIONLENS	Capital • ML
COMMISSION-ADJUSTED ALPHA PREDICTION IN MUTUAL FUNDS		
05	STRESSTEST	Capital • Finance
BANKING SECTOR STRESS TESTING & FINANCIAL RESILIENCE IN INDIA		
06	TRUSTPAY	De-Fi • Blockchain
MILESTONE-BASED DISTRIBUTED ESCROW PAYMENT PLATFORM		

REGIME-SHIFT

MACRO-AWARE TACTICAL ASSET ALLOCATION ENGINE

PROBLEM STATEMENT

Standard quantitative strategies and static portfolios (like the classic 60/40) rely heavily on stationary assumptions, working perfectly during prolonged bull runs but failing catastrophically during structural market breaks. In the institutional Global Macro and Multi-Asset space, a static allocation is a liability. The ability to mathematically detect, quantify, and adapt to unobservable economic regime changes is the absolute core of modern portfolio management.

This project bridges the gap between unsupervised machine learning and classical convex portfolio theory. You will build a dynamic allocation engine from scratch that ingests noisy, multi-asset market data, classifies hidden economic states using Hidden Markov Models (HMM), and automatically shifts capital between equities, fixed income, and safe havens.

The true challenge of this project isn't just fitting a statistical model to historical data; it is executing that logic without falling victim to look-ahead bias and portfolio thrashing. Standard backtests often hallucinate alpha by acting on data from the future or ignoring the devastating cost of high turnover. By building a rigorous walk-forward validation harness, dynamically mapping optimization objectives to specific regimes, and explicitly penalizing transaction friction, you will engineer a system designed for robust, real-world survival - demonstrating the exact risk-aware mindset that top-tier hedge funds demand from quantitative researchers.

GOALS

→ Build an HMM Regime Classifier — ingest asset returns, VIX, and macro proxies to detect hidden market states (Bull, Bear, Crisis) without manual labelling.

→ Implement Dynamic Constraint Mapping — shift optimizer objective functions by regime: Maximum Sharpe in Bull, Minimum Volatility in Crisis.

→ Build a Walk-Forward Validation Harness — strict temporal train/test splits, re-fitting the HMM only on past data, eliminating look-ahead bias.

→ Model Transaction Friction Explicitly — apply 5-10 bps turnover penalty per rebalance; show the model survives transaction costs.

→ Benchmark Against Static Portfolios — 60/40 and equal-weight baselines across Sharpe, max drawdown, and Calmar ratio.

DELIVERABLES

→ Trained HMM model with regime labels overlaid on historical price data and documented transition probabilities.

→ Dynamic allocation backtester with rigorous walk-forward validation, annotated regime charts, and equity curve visualizations.

→ Performance Tear Sheet — Sharpe, Sortino, max drawdown, Calmar ratio, and turnover analysis vs. static benchmarks.

→ Jupyter Notebook pipeline covering data ingestion → regime detection → optimization → backtest → reporting.

→ GitHub README with architecture decisions, testing instructions, and reproducibility guide.

TECH STACK

- Python 3.9+
- hmmlearn
- CVXPY
- NumPy
- Pandas
- Matplotlib
- yfinance
- scipy

CONCEPTS TO MASTER

- Hidden Markov Models & Viterbi Algorithm
- Convex Portfolio Optimization
- Walk-Forward Validation
- Sharpe / Sortino / Calmar Ratios
- Transaction friction modelling

DATA SOURCES

- Yahoo Finance (yfinance) — multi-asset daily returns
- FRED API — macro indicators (CPI, yields, spreads)
- CBOE VIX historical data

MENTORS

- Shubham Rane: 98205 59692
- Tanishq Kothari: 98318 23372

VECM-ARB

DISTRIBUTED CROSS-ASSET STATISTICAL ARBITRAGE ENGINE

PROBLEM STATEMENT

Standard pairs trading relies on identifying simple, two-asset mean-reverting relationships. However, modern financial markets are highly correlated ecosystems. In institutional quantitative trading, alpha is extracted not just from pairs, but from complex, high-dimensional baskets of assets moving together in an equilibrium state. The ability to mathematically identify these multivariate relationships and build the infrastructure to trade them synchronously is a highly sought-after skill on quantitative trading desks.

This project bridges the gap between advanced multivariate econometrics and distributed systems engineering. You will build a "Large" statistical arbitrage system that identifies cointegration in a k-dimensional space, models the speed of mean reversion, and executes dynamic hedges across a distributed, event-driven architecture.

The true challenge of this project isn't just the math, it is the system constraints. You will need to maintain market-neutrality when an asset suddenly becomes illiquid, handle non-synchronous data streams without blocking your math engine, and explicitly model the physical drag of market impact and borrow fees.

GOALS

→ Build an N-Dimensional Alpha Engine — Johansen Test across 6-8 assets, construct VECM to extract hedge ratios (β) and speed of adjustment (α).

→ Implement a Distributed Data Pipeline — Redis/ZeroMQ Pub/Sub architecture where independent microservices push order book states without blocking the math engine.

→ Develop a Dynamic Risk & Hedging Engine — if an asset faces a trading halt or becomes hard-to-borrow, recalculate optimal weights for remaining N-1 assets using convex optimization.

→ Build a Microstructure-Aware Backtester — maker/taker fees, annualized short borrow costs, and square-root market impact based on ADV.

→ Integrate Cointegration Breakdown Protocol — rolling Johansen validation; trigger automated liquidation if rank r drops.

DELIVERABLES

→ Modular codebase — separate modules for VECM math, message broker logic, and backtesting engine.

→ System Architecture Diagram showing data flow from APIs → Redis/ZeroMQ → dynarisk engine → portfolio allocator.

→ Performance Tear Sheet — Sharpe, max drawdown, and capacity analysis (at what AUM does market impact destroy alpha).

→ GitHub README documenting econometric decisions, distributed architecture rationale, and local backtest setup guide.

TECH STACK

- Python 3.9+
- statsmodels
- CVXPY
- Redis / ZeroMQ
- asyncio
- NumPy / Pandas
- scipy.optimize

CONCEPTS TO MASTER

- Johansen Test & eigenvalue interpretation
- Vector Error Correction Models (VECM)
- Distributed Pub/Sub Systems
- Convex Optimization under constraints
- Square-root market impact model
- Short borrow cost modelling

DATA SOURCES

- Yahoo Finance (yfinance) — daily adjusted close
- Binance / Bybit API — live crypto HFT data
- Alpaca / Polygon.io — minute-level sector ETFs

MENTORS

- Shubham Rane: 98205 59692
- Tanishq Kothari: 98318 23372

NANOMATCH

ULTRA-LOW LATENCY ORDER MATCHING ENGINE

PROBLEM STATEMENT

Standard algorithmic trading strategies are often written in high-level languages, focusing purely on alpha generation and mathematical models. However, in the high-frequency trading (HFT) industry, the most brilliant alpha model is completely useless if the underlying system executing the trade is a microsecond too slow. The absolute core of any financial exchange—and the primary battleground for quantitative systems engineers—is the Order Matching Engine.

This project bridges the gap between theoretical quantitative finance and hardcore, low-level systems engineering. You will build a fully functional Limit Order Book (LOB) from scratch that ingests raw market data, dynamically manages queues of buy and sell orders, and executes trades based on strict price-time priority.

The true challenge of this project isn't just implementing the matching logic, it is executing that logic with absolute hardware sympathy. Standard C++ practices, heavily utilized in competitive programming (like `std::map` or dynamic memory allocations), introduce unpredictable latency spikes due to operating system overhead and CPU cache misses. By stripping away these high-level abstractions to build cache-aligned, contiguous data structures, you will engineer a system designed for deterministic, sub-microsecond execution—demonstrating the exact architectural mindset that top-tier proprietary trading firms demand from systems engineers.

GOALS

- Build a Core Limit Order Book — process Buy/Sell limit orders, market orders, and cancellations on strict price-time priority.
- Implement Cache-Optimized Memory Management — replace STL containers with custom memory pools and contiguous arrays to eliminate OS overhead and L1/L2 cache misses.
- Develop a High-Throughput Data Ingestion Pipeline — parse raw binary PCAP/ITCH files or multi-million row CSVs using zero-copy mmap.
- Integrate Lock-Free Concurrency — asynchronous SPSC ring buffer using atomic operations for trade logging without blocking the main execution thread.
- Establish a Rigorous Benchmarking Harness — `rdtsc` / Google Benchmark, reporting throughput, p50, p90, and p99 latencies.

DELIVERABLES

- Functional C++ Matching Engine — compilable via CMake, processes datasets with strict price-time priority matching.
- Performance Benchmark Report — throughput (orders/sec), p50 median latency, p90 and p99 tail latencies; baseline STL vs. optimized comparison.
- Visual Profiling Evidence — CPU Flame Graphs (Linux perf or VTune) proving L1 cache miss resolution.
- The "HFT Narrative" README — architecture decisions, hardware specs used for testing, and benchmark reproduction instructions.

TECH STACK

- C++17 / 20
- CMake
- Google Benchmark
- Intel VTune
- Linux perf
- mmap / ITCH
- `std::atomic`

CONCEPTS TO MASTER

- Cache locality — L1/L2/L3 vs. RAM hierarchy
- Custom memory pools & arenas
- Lock-free concurrency & memory ordering
- Zero-copy I/O via mmap
- Struct packing & false sharing prevention
- Branch misprediction & pipeline hazards

DATA SOURCES

- NASDAQ TotalView-ITCH sample .pcap files
- Synthetic Order Generator — Python/AI-generated multi-million row CSV with engineered edge cases

MENTORS

- Shubham Rane: 98205 59692
- Tanishq Kothari: 98318 23372

COMMISSIONLENS

COMMISSION-ADJUSTED ALPHA PREDICTION IN MUTUAL FUNDS

PROBLEM STATEMENT

India has over 90 million demat accounts as of 2024, yet a majority of retail investors remain unaware of a critical cost embedded in mutual fund investing — the expense ratio gap between regular and direct plans. Although both plans invest in identical assets under the same fund manager, regular plans charge an additional 0.5%-1.5% annually as distributor commission. Due to compounding, this seemingly small difference can erode ₹8-12 lakh from a ₹5,000 monthly SIP corpus over 20 years. The deeper issue is that a higher expense ratio is only financially justified if the fund generates sufficient alpha — excess return over the benchmark — to offset the additional cost. If a fund earns 0.6% alpha but charges 1.1% more than its direct counterpart, the investor loses 0.5% annually despite paying for active management. In such cases, the commission is not a fee for value — it is a cost with no return. Currently, no systematic framework exists in the Indian context to evaluate whether a mutual fund will justify its expense ratio going forward. This project addresses that gap by using historical fund data and machine learning to predict whether a mutual fund will generate net positive alpha after accounting for its commission cost.

GOALS

→ Compute commission-adjusted net alpha for Indian equity mutual funds using historical NAV and benchmark return data.

→ Engineer fund-level features — expense ratio gap, information ratio, portfolio turnover, AUM, fund manager tenure, rolling Sharpe ratio, beta — as predictors of alpha sustainability.

→ Incorporate macroeconomic regime variables — RBI repo rate, CPI inflation, yield curve slope, FII/DII flows — as contextual features.

→ Train and evaluate ML models predicting next-quarter net alpha for each fund.

→ Run a binary classification task labelling each fund-quarter as commission-justified or unjustified; identify feature importance using SHAP values.

→ Back-validate using SIP simulation — compare XIRR of model-selected funds vs naive regular plan investing over 5 years.

DELIVERABLES

→ Cleaned and engineered dataset of 200+ Indian equity mutual funds with quarterly fund-level and macroeconomic features spanning 5 years.

→ Trained ML models with documented performance metrics — RMSE, AUC-ROC, F1 score, and precision at top decile.

→ SHAP explainability report identifying the 3-5 fund characteristics most predictive of commission justification in the Indian mutual fund market.

→ SIP back-validation analysis comparing XIRR of model-guided fund selection versus naive regular plan investing over 2019-2023.

→ Jupyter notebook pipeline covering the complete workflow from data collection to model evaluation, submitted as executable code and written research report.

→ Optional: Streamlit dashboard where a user inputs a fund name and receives its predicted commission-justification score for the current quarter.

TECHNOLOGY STACK

- Python
- XGBoost
- scikit-learn
- SHAP
- Pandas / NumPy
- scipy (XIRR)
- Streamlit
- Matplotlib / Seaborn

DATA SOURCES

- AMFI public NAV API
- mfapi.in — Indian mutual fund data
- RBI DBIE — repo rate, CPI, yield curve
- NSE historical data — benchmark returns

CONCEPTS TO MASTER

- Net Alpha = Gross Return - Benchmark - Expense Gap ♦
- Information Ratio & alpha persistence
- XIRR for SIP cash flows
- Temporal train-test split (no look-ahead bias)
- SHAP feature importance
- Rupee Cost Averaging & market regimes
- Portfolio Turnover & hidden trading costs

MENTOR

- Harshita Sharma: 98276 15327

STRESSTEST

BANKING SECTOR STRESS TESTING & FINANCIAL RESILIENCE IN INDIA

BACKGROUND

The Indian banking system plays a central role in economic stability, acting as the primary channel for credit distribution, liquidity transmission, and financial intermediation. However, banks are inherently exposed to macroeconomic shocks such as economic slowdowns, rising interest rates, and credit defaults. Events like rising NPAs, liquidity crises, and global tightening cycles have highlighted the need for robust stress-testing frameworks to assess the resilience of banks under adverse conditions.

YOUR CHALLENGE

As a team of financial analysts and risk consultants, your task is to develop a structured stress-testing framework to evaluate how Indian banks would perform under different macroeconomic stress scenarios. Your framework should assess the impact of shocks on profitability, asset quality, and capital adequacy, while comparing resilience across different categories of banks. You are required to select four representative banks for your analysis – two from the public sector and two from the private sector – such that the selection creates a meaningful contrast in risk profile, capital strength, and operational efficiency.

CORE FOCUS AREAS

1. Risk Identification & Categorization

- Credit risk, interest rate risk, liquidity risk.
- Categorize risks by severity and likelihood.

2. Scenario Design & Stress Assumptions

- Baseline: current macro conditions.
- Moderate Stress: COVID-19 FY2020-21.
- Severe Stress: 2015-16 NPA crisis plus sharp rate hike.
- Define NPA, NIM, CAR and ROA/ROE assumptions.

CORE FOCUS AREAS CONT.

3. Financial Impact Analysis

- NIM, net profit, ROA, ROE, GNPA, NNPA, PCR, CAR.

4. Resilience Scorecard

- Capital buffer, NPA absorption capacity, NIM sensitivity, liquidity buffer, severe-stress profitability.

5. PSU vs Private Banks

- Risk exposure, capital buffers, efficiency, implicit government support, market discipline.

CONSTRAINTS

- Use only publicly available financial data – annual reports, RBI publications, BSE/NSE filings.
- Focus on 4 banks: 2 PSU and 2 private sector.
- Build an assumption-driven model; all inputs must be documented and auditable.
- No complex econometric modelling is required.
- The framework must be intuitive and scalable.

GUIDING QUESTIONS

- Which risk factor has the highest impact on bank stability under stress – credit risk, interest rate risk, or liquidity risk?
- Are private banks structurally more resilient than PSU banks, or does implicit government backing offset PSU vulnerabilities?
- How sensitive are bank profits to a 200 bps increase in NPAs versus a 100 bps rise in interest rates?
- Do current capital buffers adequately absorb severe shocks without breaching regulatory minimums?
- What early warning indicators would have signalled the 2015-16 NPA crisis ahead of its peak?

STRESSTEST

BANKING SECTOR STRESS TESTING & FINANCIAL RESILIENCE IN INDIA

DELIVERABLES

→ A stress-testing model simulating three scenarios across all selected banks.

→ A resilience scorecard ranking banks by composite stress resilience.

→ An early warning indicators dashboard for each bank.

→ A comparative analysis of PSU vs. private banks under stress.

→ Bank-specific strategic recommendations grounded in the stress model findings.

→ A concise report or presentation not more than 12 pages/slides covering methodology, results, and implications.

DATA SOURCES

- Annual Reports – selected banks
- RBI Publications & DBIE database
- BSE / NSE regulatory filings
- RBI Financial Stability Reports

MENTOR

- Manit Mehta: 89208 77266

EXPECTED OUTCOME

How resilient is the Indian banking system to macroeconomic shocks, and where do the key vulnerabilities lie?

TRUSTPAY

MILESTONE-BASED DISTRIBUTED ESCROW PAYMENT PLATFORM

CONTEXT

Here's a problem you've probably experienced personally: you hire someone online, you're nervous to pay upfront because they might disappear, they're nervous to start work because you might not pay. The traditional solution is an escrow service — a middleman who holds the money and releases it when both parties agree. But traditional escrow is slow, expensive, and ultimately just another party you have to trust. What if the escrow was a piece of code that anyone could read, running on a network that nobody controls?

PROBLEM STATEMENT

The smart contract is ~80 lines of Solidity and serves one precise role: holding funds without a trusted third party. The backend is where 90% of the engineering effort lives — state machine design, concurrency control, async job queues, fault-tolerant recovery, and audit trails. These are the same distributed systems problems that FAANG engineers solve in production every day.

Every escrow progresses through a defined lifecycle: `CREATED` → `FUNDED` → `IN_PROGRESS` → `SUBMITTED`, then either `RELEASED`, back to `IN_PROGRESS`, or `DISPUTED` → `RESOLVED` via admin arbitration.

The system must stay consistent when things go wrong — when the blockchain confirms a payment but the server crashes before the database updates, or when two admins attempt to resolve the same dispute simultaneously.

GOALS

→ Build a Correct Escrow State Machine: Design and implement the complete escrow lifecycle — `CREATED` → `FUNDED` → `IN_PROGRESS` → `SUBMITTED` → and then branching into `APPROVED` → `RELEASED`, `REJECTED` → `IN_PROGRESS`, or `DISPUTED` → `RESOLVED`. Every transition must enforce strict preconditions, execute defined side effects, and handle the failure case where one side effect succeeds and the other does not.

→ Implement the Distributed Transaction Pattern: Solve the partial failure problem using an async job queue (BullMQ), pending transaction hash tracking in the escrow record, and a background confirmation worker that drives state updates idempotently. A state transition is not complete until both the on-chain event and the off-chain database agree.

→ Build Concurrent-Safe State Management: Prevent race conditions such as double-release or simultaneous dispute initiation by implementing distributed locking via Redis (Redlock pattern) on the escrow ID before any state transition. Use optimistic locking at the database layer as a fallback.

→ Design a Full Append-Only Audit Trail: Implement an immutable audit log in PostgreSQL that records every state transition — the previous state, the new state, the actor's identity and role, the timestamp, and the blockchain transaction hash where applicable. This table accepts inserts only. No updates. No deletes.

CONCEPTS TO MASTER

→ The Distributed Transaction Problem (Two-Phase Commit): When a single action requires writing to a blockchain and a database atomically, you cannot use a traditional transaction. Learn to solve this using the Saga pattern with a background confirmation worker.

→ State Machine Design and Transition Authorization: A state machine is more than a field storing a string — it is an explicit declaration of which transitions are legal, who can trigger them, and what side effects must happen atomically.

→ Distributed Locking and Concurrency Control: The Redlock algorithm uses atomic operations across Redis instances to acquire a distributed lock on a resource identifier before modifying it.

→ Idempotency in Payment Systems: Every operation that can be retried must be idempotent — executing it twice must produce the same outcome as executing it once.

TRUSTPAY

MILESTONE-BASED DISTRIBUTED ESCROW PAYMENT PLATFORM

REMAINING GOALS

- Write a Minimal, Secure Smart Contract: Build a Solidity escrow contract that holds and releases funds using the ReentrancyGuard pattern and the Check-Effects-Interactions principle. The contract must emit an event for every state-changing function. All business logic lives off-chain. The contract is purely a fund vault.
- Implement a Reputation Engine: Build a post-contract scoring system that computes a numeric trust score for every user, weighted by contract value, on-time delivery rate, and dispute frequency. Reputation scores must be queryable, displayed in the application, and updated asynchronously via a job queue after contract resolution.
- Deliver a Production-Grade API and Frontend: Expose a clean REST API with JWT authentication, proper authorization middleware, and structured error responses. Build a functional React frontend with wallet connection via wagmi and RainbowKit, real-time WebSocket notifications for transaction confirmations and milestone updates.

DELIVERABLES

- Functional Full-Stack Application: A deployable web application covering the complete user journey — registration and wallet connection, job posting and freelancer application, contract creation with on-chain fund locking, milestone submission and approval, dispute initiation, and reputation display. The application must function correctly on the Polygon Amoy testnet with real wallet interactions. A demo with mocked transactions does not qualify.
- Deployed and Verified Smart Contract: A Solidity escrow contract deployed to the Polygon Amoy testnet with the contract address documented, source code verified on PolygonScan, and a Hardhat test suite covering all state transitions and security edge cases (reentrance attempt, unauthorized caller, double-release). The test suite must pass with documented coverage.
- System Architecture Document: A written document describing the escrow state machine with all valid transitions and their preconditions; the distributed transaction flow with sequence diagrams; the database schema with entity relationships; and the concurrency strategy explaining where and why locks are acquired.
- Engineering README: A professional GitHub repository README articulating the problem being solved, key architectural decisions and tradeoffs, specific engineering challenges encountered (distributed transaction problem, concurrency issue, reentrancy risk) and how they were addressed, and instructions to run the project locally.
- Test Suite: Unit tests covering the escrow state machine logic, integration tests for critical API endpoints, and smart contract tests in Hardhat covering normal operation and adversarial edge cases.

MILESTONE TIERS

- Tier 0 - Beginner — New to Blockchain: The contract is given to you as a complete Hardhat project (contracts/TrustPay.sol, deploy script, 8 smoke tests, .env.example). Your job is to deploy it and wire your backend and frontend around it. Think of it like being given a Stripe SDK. The engineering challenge is entirely in the backend and frontend.
- Tier 1 - Core — Required for Full Evaluation: Write TrustPay.sol from scratch. Implement fundEscrow, releaseFunds, raiseDispute, and refund functions with proper require() guards, ReentrancyGuard, and events. Write a full Hardhat test suite. Pull state machine logic into a dedicated module. Add escrow-level authorization middleware.

TECH STACK — BACKEND

- Node.js / TypeScript
- PostgreSQL
- Redis (Redlock)
- BullMQ
- ethers.js / viem

TECH STACK — SMART CONTRACT & FRONTEND

- Solidity 0.8.x
- Hardhat
- OpenZeppelin
- Polygon Amoy
- React + Vite
- wagmi + RainbowKit
- TailwindCSS
- Socket.io

MENTOR

- Kumar Utkarsh 92631 79322